XRComm FCS Wideband Receive Platform

Quick Start & User Guide

XRComm Inc.

June 2026

XRComm FCS Wideband Receive Platform

Quick Start & User Guide

Platform	XRComm FCS Wideband Receive Platform
Driver Version	v1.4.3
Variant	Wideband
Audience	End users and system administrators
Classification	Proprietary

Contents

Terminology

The following terms are used throughout this document.

Term	Meaning
FlexServer	The physical host machine running the XRComm platform driver and gRPC control service. All platform binaries execute here.
Client Computer	Any machine (including the FlexServer itself) from which you run the Python gRPC control client to operate the platform.
Platform Driver	The `xrcomm-server` binary. Receives the high-speed RF stream from the NIC, manages shared memory, records to NVMe, and dispatches data to customer applications.
Control Service	The `xrcomm-grpc-ctrl` binary. Provides a gRPC interface (TCP port 50051) for remotely controlling the Platform Driver.
Customer Application	Your own software (or the provided Hello World examples) that connects to the Platform Driver via shared memory to consume IQ samples or full packets.
IQ Delivery	Mode in which interleaved int16 I/Q sample batches are dispatched to connected applications via zero-copy shared memory.
Full-Packet Delivery	Mode in which raw network packets are forwarded to a single application using DPDK secondary process attachment. Requires `sudo`.
NVMe Recording	Mode in which the received RF stream is written directly to the NVMe drive via SPDK for later offline extraction.

System Topology

The platform operates as a set of background services on the FlexServer. Remote or local clients can control the system over the network using a Python gRPC client.

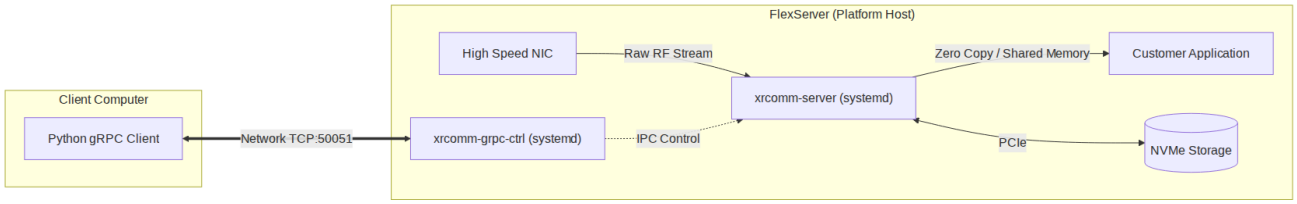

Figure 1: XRComm FCS Wideband Receive Platform Topology

Data flow:

1. The **High Speed NIC** receives the raw RF stream and delivers it to the **Platform Driver** via DPDK.
2. The **Platform Driver** dispatches IQ samples to connected **Customer Applications** via zero-copy shared memory.
3. Simultaneously, the Platform Driver can record the stream to the **NVMe Storage** via SPDK.
4. The **Control Service** communicates with the Platform Driver over IPC, allowing remote management.
5. The **Python gRPC Client** (on the **Client Computer** or on the FlexServer itself) connects to the Control Service over TCP port 50051.

Waveform Playback System Topology

The playback system acts as a signal source, cabled directly to the receive ports of the FlexServer.

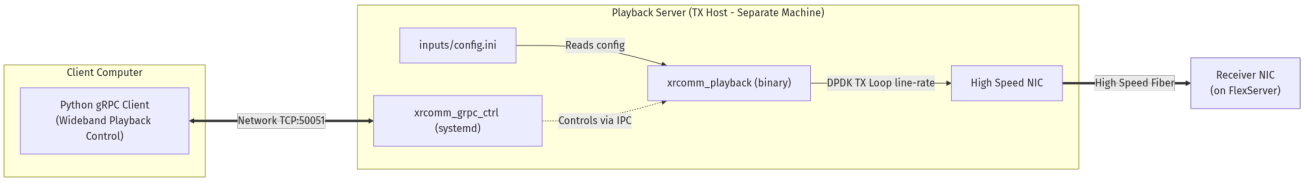

Figure 2: XRComm FCS Wideband Playback Topology

[!IMPORTANT] **Playback Hardware Separation:** The Playback Engine (`xrcomm_playback`) operates as a high-rate transmitter (pacing IQ samples to DPDK at wire-rate). To prevent host resource contention (such as CPU context switches, PCIe bus saturation, or memory channel exhaustion) with the platform receiver on the FlexServer, **the Playback Engine and its Control Server must run on a separate server machine**. Do not execute the playback engine on the FlexServer in production.

Package Contents

After extracting the tarball, you will find the following directory structure:

```
XRComm_FCS_platform/
  install.sh                    # Automated installer
  uninstall.sh                  # Clean uninstaller
  LICENSE                       # Software license agreement
  VERSION                       # Release version identifier
  CHANGELOG.md                  # Release notes
  Wideband_Quickstart.pdf       # This document

  XRComm_FCS_wideband_platform/
    XRComm_platform_drivers/
      bin/                      # Precompiled platform driver binary
      include/                  # Public C headers for app development
      service/                  # Systemd unit file templates
    XRComm_platform_gRPC/
      bin/                      # Precompiled gRPC control service binary
      client/                   # Python gRPC client scripts
      proto/                    # Protobuf service definition (.proto)
      service/                  # Systemd unit file templates

  XRComm_wideband_hello_world/
    hello_world.c               # IQ delivery example (no sudo)
    hello_world_fullpkt.c       # Full-packet delivery example (sudo)
    CMakeLists.txt              # Build configuration
    bin/                        # Output directory for compiled examples

  XRComm_wideband_playback/
    bin/                        # Precompiled playback binary
    inputs/                     # Playback config and waveform files

  XRComm_NVMe_loader/
    bin/                        # Precompiled NVMe reader binary
    config/                     # Read-back config (read_config.ini)
    scripts/                    # Interactive extraction (read_nvme.sh)
```

Installation

Extract the Package

Transfer the tarball to your FlexServer and extract it:

```
tar -xzvf XRComm_FCS_platform.tar.gz
cd XRComm_FCS_platform
```

Run the Installer

```
sudo ./install.sh
```

When prompted, accept the license agreement and select **Option 1** for the Wideband Platform.

The installer performs the following actions automatically:

Step	What it does
Dependency installation	Installs `python3-grpcio`, `python3-protobuf`, and `python3-grpc-tools` via `apt`.
Systemd service creation	Creates and enables `xrcomm-server.service` and `xrcomm-grpc-ctrl.service`.
Header file mapping	Symlinks the public C headers into `/usr/include/xrcomm-wideband/`.
Proto compilation	Generates the Python gRPC stubs (`pipeline_control_pb2.py`) in the client directory.
Service startup	Starts both services immediately.

Uninstallation

```
sudo ./uninstall.sh
```

This stops and disables the systemd services, removes the header symlinks, and uninstalls the gRPC Python packages.

Verifying the Services

After installation, both platform services run in the background. Verify their status:

```
systemctl status xrcomm-server.service
systemctl status xrcomm-grpc-ctrl.service
```

Both should report `active (running)`.

To inspect live logs:

```
journalctl -u xrcomm-server.service -f        # Platform Driver logs
journalctl -u xrcomm-grpc-ctrl.service -f      # Control Service logs
```

To restart the services after a configuration change:

```
sudo systemctl restart xrcomm-server.service
sudo systemctl restart xrcomm-grpc-ctrl.service
```


Controlling the Platform over gRPC

The remote control client is designed to run from the **Client Computer**.

Endpoint

The Control Service listens on `0.0.0.0:50051` by default. It starts automatically as a systemd service and does not need to be launched manually.

Client Setup

The Python gRPC client can be run from **any machine** (specifically the **Client Computer**) that has network access to the FlexServer:

- **From the FlexServer itself** (via loopback `127.0.0.1`) for internal verification during installation.
- **From a remote Client Computer** for standard operational use.

Install the Python dependencies once per machine:

```
cd XRComm_FCS_wideband_platform/XRComm_platform_gRPC/client
pip3 install -r requirements.txt
```

The installer already generates the protobuf Python bindings on the FlexServer. If you are setting up the client on a separate machine, copy the `client/` directory there and generate the stubs:

```
python3 -m grpc_tools.protoc -I. --python_out=. \
    --grpc_python_out=. pipeline_control.proto
```

Control Commands

Usage syntax:

```
python3 xrcomm_grpc_client.py --host <server-ip>:50051 <command> [value]
```

Replace `<server-ip>` with the IP address of your FlexServer (use `127.0.0.1` when running locally).

Command	What it does
`get-flags`	Show current delivery and recording mode flags.
`set-ip on|off`	Turn IQ-sample delivery to applications on or off.
`set-dsp on|off`	Turn full-packet delivery to applications on or off.
`set-logging on|off`	Start or stop NVMe recording. Stopping triggers automatic read-back (Section 7).
`set-read-capture <seconds> [--start-offset <sec>]`	Set read-back duration (seconds) and optional start offset from capture start (seconds).
`set-read-rate <hz>`	Set the receiver sample rate, in Hz, used to size the read-back.
`get-stats`	Show pipeline counters and platform telemetry.
`get-mode-stats`	Show received/processed/dropped counts per delivery mode and per application.
`list-ips`	List the applications currently connected to the platform.
`watch-status`	Stream live status once per second (Ctrl-C to stop).
`shutdown`	Gracefully shut the platform down.

Worked Examples

```
C="python3 xrcomm_grpc_client.py --host 192.168.137.217:50051"

$C get-flags                                # Read current modes
$C set-ip on                                # Start IQ delivery
$C set-dsp on                               # Start full-packet delivery
$C set-read-rate 800000000                  # Set receiver rate for read-back sizing
$C set-read-capture 1.5 --start-offset 0.5  # Set readback (1.5s duration, 0.5s start offset)
$C set-logging on                           # Begin recording to NVMe
$C set-logging off                          # Stop recording -> auto read-back starts
$C list-ips                                 # Confirm your application is connected
$C get-stats                                # Pipeline + platform telemetry
$C get-mode-stats                           # Per-mode and per-application counters
$C watch-status                             # Live 1 Hz stream (Ctrl-C to stop)
$C shutdown                                 # Graceful shutdown
```

Telemetry Fields

The `get-stats` and `watch-status` commands report platform telemetry:

Field	Meaning
`hsp_port`	High-speed receive port in use.
`samples_received`	Total samples received from the interface.
`samples_logged`	Samples written to NVMe (zero unless recording).
`samples_processed_secondary`	Samples consumed by your application(s).
`packets_dropped`	Receive-overflow and app-backlog drops, combined.

Mode Rules

- The three delivery/recording modes (IQ, full-packet, and recording) are independent and may be enabled in any combination.
- Full-packet delivery serves one application at a time. IQ delivery may be shared by multiple applications.

Recording Captures and Reading Them Back

Automatic Read-Back Configuration

Three values control the automatic read-back. They are set from the control client and stored in `XRComm_NVMe_loader/config/read_config.ini`:

Key	Default	Meaning
`read_duration_seconds`	`1.0`	Seconds of the most recent capture to copy back when recording stops.
`sample_rate_hz`	`800000000`	Receiver sample rate (Hz), used to convert duration into data size.
`start_offset_seconds`	`0.0`	Offset (in seconds) from the start of the capture at which to begin.

The amount copied back is `seconds x sample_rate x 4 bytes` (each sample is one little-endian int16 I value followed by one little-endian int16 Q value), rounded to the nearest whole record and capped to the capture length. Example: 1 second at 800 MHz copies approximately 3,051.76 MiB.

Setting Read-Back Parameters via gRPC

The parameters are configured from the **Client Computer** running the gRPC client:

```
# Set the read-back duration to 1.5 seconds and start offset to 0.5 seconds
python3 xrcomm_grpc_client.py --host <server-ip>:50051 set-read-capture 1.5 --start-offset 0.5

# Set the receiver sample rate to 800 MHz
python3 xrcomm_grpc_client.py --host <server-ip>:50051 set-read-rate 800000000
```

Recording Workflow

```
python3 xrcomm_grpc_client.py --host <server-ip>:50051 set-logging on   # Start
python3 xrcomm_grpc_client.py --host <server-ip>:50051 set-logging off  # Stop -> auto read-back
```

Output Files

Read-back files are written under the Platform Driver's working directory in `output_logs/readback_<timestamp>/`:

- `readback_<timestamp>_chunk_N.bin` – raw interleaved int16 little-endian IQ pairs (I, Q, I, Q, ...), split into 2 GB chunks.
- `readback_<timestamp>.idx` – a text manifest describing each block in the `.bin` files.

Offline NVMe Retrieval

To extract older or specific data blocks manually after the platform has stopped:

```
cd XRComm_NVMe_loader/scripts
sudo ./read_nvme.sh
```

The underlying binary (`xrcomm_nvme_reader`) also accepts options for partial extraction:

Option	Effect
`--first-mb=<N>` / `--first-gb=<N>`	Extract the first N megabytes / gigabytes.

Option	Effect
`--middle-mb=<N>` / `--middle-gb=<N>`	Extract from the middle of the capture.
`--last-mb=<N>` / `--last-gb=<N>`	Extract the last N megabytes / gigabytes.
`--flash`	Free the capture on the drive after successful full extraction.
`--force-flash`	Force delete even if extraction is partial or corrupted.

Extracted data uses the same format as the automatic read-back.

Waveform Playback Platform

The Wideband variant includes the Waveform Playback Engine (`XRComm_wideband_playback/`) for streaming pre-recorded IQ waveforms over the network interface.

Playback Directory Structure

The precompiled playback engine and its remote control server reside in the following directory layout:

```
XRComm_wideband_playback/
├── bin/
│   ├── xrcomm_grpc_ctrl                # Precompiled control server
│   └── xrcomm_playback                 # Precompiled playback engine
├── grpc/
│   ├── client/
│   │   ├── README.md
│   │   ├── XRComm_wideband_playback_client.py # Example client control
│   │   ├── pipeline_control_pb2.py       # Shipped precompiled stub
│   │   └── pipeline_control_pb2_grpc.py  # Shipped precompiled stub
│   ├── proto/
│   │   └── pipeline_control.proto        # Service contract
│   └── server/
│       ├── bin/
│       └── xrcomm_playback_control.cc    # C++ server implementation
└── inputs/
    ├── config.ini                        # Playback configuration
    └── playback_waveforms/
        ├── binary_search_tool.py         # Waveform helper
        ├── hexdump
        ├── split_bin.py                  # Stream splitter helper
        └── tone_512.bin                  # Sample waveform
```

Playback Configuration Keys (`inputs/config.ini`)

Start parameters are loaded from `inputs/config.ini` at startup.

Key	Default	Meaning
`core_list`	`7,8,9`	DPDK lcore IDs to assign to the playback engine (isolate these cores)
`source_mode`	`1`	`1` = waveform file playback, `0` = streaming from NVMe drive capture
`binary_filename`	`tone_512.bin`	Waveform file resolved under `inputs/playback_waveforms/`
`target_sample_rate_hz`	`800000000`	Sample rate in Hz (e.g. `800000000` = 800 MHz)
`tx_port_id`	`0`	Physical DPDK network port ID to transmit from

Key	Default	Meaning
`dest_mac`	`14:FE:B5:DD:9A:82`	Destination MAC address for transmitted packets
`loop_count`	`0`	`0` = infinite loops, `N` = stop after N loops
`trigger_burst_size`	`1`	Number of packets marked active per 'T' console trigger

Running the Playback Control Server

Start the control server from the `XRComm_wideband_playback/` directory on the **Playback Server**:

```
export XRCOMM_PLAYBACK_ROOT=$(pwd)
./bin/xrcomm_grpc_ctrl 0.0.0.0:50051
```

Running the Playback Client (Client Computer)

Operate the playback remotely from the **Client Computer** using the precompiled Python stubs:

```
cd XRComm_wideband_playback/grpc/client
python3 XRComm_wideband_playback_client.py --endpoint <playback-server-ip>:50051 <command> [args]
```

Playback gRPC Client Commands

- `get-config` - Read the 8 current `config.ini` parameters.
- `get <field>` - Read a single configuration parameter.
- `set <field> <value>` - Set a single configuration parameter (takes effect next run).
- `start` - Launch the playback engine on the Playback Server.
- `stop` - Stop the running playback engine cleanly.
- `diag` - Return a single diagnostics snapshot (running time, transmit rate, etc.).
- `watch` - Stream live diagnostics once per second (Ctrl-C to stop).

Runtime Console Controls (Playback Server)

If running the playback engine (`xrcomm_playback`) directly or monitoring its console, use the following keys: - `T` or `t` : Arm a trigger burst. Marks the next `trigger_burst_size` packets as active, beginning at the next waveform loop boundary. - `+` / `-` : Fine pacing adjustment (± 1 Hz step). - `]` / `[` : Coarse pacing adjustment (± 10 Hz steps). - `#<number>` : Configure pacing tuning rate directly in Hz (e.g. `#20000`). - `q` / `Ctrl+C` : Gracefully stop playback.

Troubleshooting

Symptom	What to check
No free hugepages on startup	Clear stale files and reset the reservation (see Appendix A).
Platform fails to start / no interface	Confirm NIC and NVMe binding (Appendix A). Check `systemctl status xrcomm-server`.
Application receives no data	Confirm the platform is running and the matching mode is enabled. Check `list-ips`.
Full-packet example will not attach	Start the platform first. Run only one full-packet application at a time.
Permission denied	Ensure platform services are running. Try running the application with `sudo`.
Control client cannot reach the service	Check `systemctl status xrcomm-grpc-ctrl`. Verify port 50051 is not blocked by a firewall.
Read-back produced no files	Only triggers on `set-logging off` after a real capture. Confirm `set-read-rate` and `set-read-capture` were set.
Components disagree after update	Re-run `install.sh` and restart all services so they share the same interface version.

Appendix A: Environment Prerequisites

These configurations are **prerequisites** for the FlexServer. They are typically set up once during initial server provisioning. If the platform fails to start or behaves unexpectedly, verify these settings first.

Dependency Version Check Table

Dependency	Required Version	Installation Command / Source
Ubuntu Linux	22.04 LTS or 24.04 LTS (64-bit)	Operating System standard install
DPDK	v23.11.1	`sudo apt install dpdk libdpdk-dev`
SPDK	v24.01	Build from source
gRPC & Protobuf	v1.62.0 (C++ runtime)	`sudo apt install libgrpc++-dev libprotobuf-dev`
Python 3	v3.10+	Operating System standard install
grpcio & tools	v1.62.0 (Python runtime)	`pip3 install grpcio grpcio-tools`
CMake	v3.16+	`sudo apt install cmake`
GCC	v11+	`sudo apt install build-essential`

Hugepages

File-backed hugepages are required for zero-copy memory pools:

```
sudo rm -rf /dev/hugepages/* /mnt/huge/*          # Clear stale allocations
sudo sysctl -w vm.nr_hugepages=0
sudo sysctl -w vm.nr_hugepages=112                # Adjust to your memory layout (112 x 2MB)
grep -i huge /proc/meminfo                         # Verify
```

Device Binding (DPDK / SPDK)

Before launching the server, the receive NIC interfaces and NVMe storage drive must be bound to the correct DPDK/SPDK user-space drivers:

- **Network Interface (DPDK Binding):** Bind the wideband receive NIC ports to the DPDK-compatible driver (`vfio-pci`) using the helper script:

  ```
  sudo modprobe vfio-pci
  sudo dpdk-devbind.py --bind=vfio-pci 0000:6a:00.0 0000:6a:00.1
  ```

- **NVMe Drive (SPDK Binding):** Bind the recording NVMe PCIe drive to SPDK using the SPDK `setup.sh` script.

Privileges

- The Platform Driver and the full-packet example require `sudo` (hugepages and direct device access).
- The IQ example and the control client do not require elevated privileges.

Support: For assistance, contact XRComm Inc. using the support address provided with your delivery package. Include the output of `systemctl status xrcomm-server`, `journalctl -u xrcomm-server.service --no-pager -n 50`, `get-stats`, and `get-mode-stats`.